Design memo

1. Retriever. The index is BM25 over subject and body. Tokens are lowercase runs of letters and digits, so there are no phrases and "$28,500" splits. Parameters are k1 = 1.5, b = 0.75, and Lucene IDF. The score is a sum over query terms, so two rare names beat a rare name plus a common word. On "Solheim Hollyoak fee" the Hollyoak schedule scores 30.9 and the engagement email with the $28,500 fee scores 21.6. The schedule names Hollyoak and Solheim and states no fee. The fee email's subject is "engagement terms"; the body names Solheim and the fee and never says Hollyoak. "fee" is common, and a Datek fee notice is fifth. The ranking is wrong for "what is the fee": the email that answers it is second, because the terms are scored independently. "margin call Orlin Valley" ranks the right email first (em-08125, $4.2 million, 2:00 PM). Second is a Valley Electric credit note that shares "valley" and the word "margin" inside "margin rights." Orlin does not appear there.

2. Tool-call protocol. The tests round-trip through format and parse and do not fix a syntax. The live prompt is one fixed sentence plus the rendered tool text, so the format has to be taught there. The live requests use seconds of 0.5, an integer n_lines, and enum strings. JSON keeps 0.5 a number and true a boolean. A key=value line would turn both into strings and fail those requests. One object, with a string name and an arguments object, sits between tool_call tags. The parser runs raw_decode from the first tag. It does not find the end by searching for the closing tag, because an argument string can contain a complete call and a tag search would split it. A cut inside the JSON is an error, so a truncated argument cannot come back as a different call. A second object in the same reply is an error, because the loop runs one tool and then asks the model again. The live model often wrote the object and skipped the tags, so a bare object with a name and an arguments object is accepted. An object that holds only arguments, such as a query, names no tool. That reply is sent back as malformed.

3. Termination. A reply with no call is the final answer. The Problem 7 scripts require that, so the loop cannot treat every no-call reply as another turn. The live model then stopped on "I'll look that up," which is not an answer. Before any tool result is in the transcript, that wording, "let me," "unable to," "can't check," an empty reply, and a JSON object of only arguments are sent back as malformed. The third in a row ends the send. After a tool has run, the same phrases stay legal, so an answer may quote "I will." "going to" was in that list and then removed: "the crew is going to be on site" is a real answer and was being rejected. "unable to" was added after a persona answered "I'm unable to provide the count" with no tool and the send ended done. "Can't save" is still a final answer. That is why the team fact in section 7 was never written.

4. Document search. Indexing a whole document would rank the right file and still hand back a snippet that missed the sentence. Each non-empty line is a passage. The first snippets were that line alone. Every line in this corpus is under 150 tokens, and the unique-word test needs four consecutive words of overlap, so a short line is grown to a neighbor until it has four words, then forward to about 150 tokens. A line over 150 is clipped around the first query term. Raw BM25 let a common word bury the line that held the rare name. Search now takes a pool of eight times k and keeps lines that contain the rarer half of the query terms. That is why the 2026 office-days announcement outranks the handbook, which still says two days. read_doc returns the exact lines from start_line, packed to 1,500 tokens, and names the next start line. The handbook is about 7,000 tokens, so one window cannot hold it, and the next read starts at end_line + 1 with no gap. The result tells the model to call read_doc when the quote is not the answer, and to use the latest date in the id or title when documents disagree.

5. Terminal observation. Under 800 tokens the model sees stdout, stderr, and the exit code. Over that, the full stdout is saved as terminal-output-N.txt and the model sees a head of about 400 tokens. Eight hundred is about two observations beside the system prompt in a 6,000-token context. A nonzero exit, a timeout, or an allowlist rejection is a tool error and includes the sandbox's reason, so the model can retry. The first large-output text also reported the file's line count. The August 2001 command printed 7,965 senders and then "Count: 7965." The saved file had 7,966 lines, the observation said so, and the model copied 7,966. The line count was removed. The figure the question wanted was the last line, which was not in the head, so the observation now adds that line when it is at most 200 characters and not already shown. A later command can read any other line from the saved file.

6. Compaction. fit runs only when the messages sent to the model exceed the budget. The transcript is unchanged, so a short session still shows the full tool result on the next call. The oldest message that is not the system prompt, the notes, or the current user text is dropped. Summarizing every dropped message would send tool results through the auxiliary model and would change how many times the agent's model is called. The tests require those results to be dropped with no model call. Only a previous user request, matched by its exact text, is summarized. The auxiliary prompt says to copy numbers, amounts, cities, and constraints. It still paraphrased. Any source sentence whose number or capitalized word is missing from the summary is appended. The notes are one message with a fixed first line, so the next summary replaces them. A single message larger than the budget, such as a 30,000-token listing, is replaced by a head that fits. On the live run the largest call was 3,995 of 4,000 tokens, and $18,450 and

Boulder stayed.

7. Memory. show memory on runs/sim-20261031-140634 leaves eighteen personas empty. memory-the-spec holds the user's sentence that "the spec" means the GridLens API spec. memory-corrected-number holds "User's employee number is 30918." and "User corrected their employee number: it is 31908, not 30918." memory-my-team is empty. The empty stores are the right outcome. Those personas never asked to keep a fact, and the file is not a log of the conversation. The spec sentence is the right thing to store, and storing the user's words is the right form: a paraphrase could drop GridLens. It was saved because the first reply was empty and the message said "remember," so the session wrote the user text itself. Both employee numbers belong on disk. The file is append-only, and the correction is meaningful next to the number it replaces. The prompt hides 30918, which is why session 2 answered 31908 alone. Leaving the team name unstored does not make sense. Session 1 was "For future reference: I'm on the Customer Success team, based in Rotterdam," and the user asked again to remember it. The model replied that it cannot save a persistent memory, called no tool, and the send ended. The empty-reply saver did not run, because the reply was not empty. Session 2 had nothing to inject and asked which team.

8. Approval. In confirm mode the loop asks only when a user is set and read_only is false. In the Cardinal set that is run_terminal. Search, read, the digest, email QA, remember, recall, and ask_user do not ask. A search or a read does not change the workspace, send mail, or open the network. remember appends one line to the agent's own facts file, which is the write the user just requested. ask_user only prompts. Stopping for those would block the ordinary questions and would not prevent a side effect those tools cannot cause. run_terminal can rewrite the workspace, so it waits. A denial is not executed. The result includes the user's reason and says the action was not run. An earlier result said only that the user declined, and the model told the user the file had been deleted. Deny rules are matched against run_terminal before approval and before the command starts, so a planted command never reaches the sandbox.

9. Sandbox. sandbox.py is the course file and was left as written. It has four layers because each earlier layer leaves a hole. The allowlist never starts rm or curl; both were rejected before a process existed. It does not look at arguments, so python3 -c is allowed, and awk, find -exec, and xargs can start other programs. The environment drops the parent variables, so no API key is visible, and PATH points only at the allowed programs. An absolute path is outside that layer. The timeout kills the process group and keeps at most 4,000,000 bytes per stream. A process that calls setsid leaves the group, and memory, CPU, and disk are not capped. On this machine jail() is sandbox-exec. python3 could not read the home directory, could not write /tmp, and could not resolve example.com. It could write a file in the workspace. An approved python3 -c can still rewrite emails.jsonl there. If jail() were none, that same command could use the network and any file the user can. This Mac does not use bwrap.

10. Limits. 1. Terms are scored independently, so the fee email ranks behind the schedule.
2. Compaction drops tool results with no summary, so a long search can lose the passage that had the answer.
3. "Can't save" is still a final answer, so a fact the user asked to keep can be missing from the next session.
4. Three empty replies end the send.
5. The model describes an action as finished unless the tool result says it was not run.
6. The meal-limit persona still states both $75 and $95.
7. An approved python3 command can rewrite the workspace copy. The allowlist does not stop it.
8. Tool descriptions were left short so earlier passing conversations stayed cached.